

Graphs, Lineage and How datagraph Was Built

A beginner's book — explained so simply that a 10-year-old can follow, yet complete enough to understand the real code.

What you will learn: what a graph is · arrows and hops · how a computer stores and searches a graph · what “data lineage” means · how datagraph reads Python, dbt, SQL and git to build its graph · how it decides what breaks and how risky a change is · which tools and skills were used to build it · exercises with answers.

Companion project: datagraph v0.2 — `C:\Users\Hp\Documents\Innovation\datagraph`

Written with Claude Code · August 2026

Contents

How to read this book.....	3
PART A — Graphs from the very beginning	4
Chapter 1 — Dots and lines.....	4
Chapter 2 — Arrows: when direction matters	5
Chapter 3 — Paths, hops and “can I get there?”	6
Chapter 4 — How a computer keeps a graph	6
Chapter 5 — The ripple game: Breadth-First Search.....	8
Chapter 6 — The domino secret: arrow direction vs impact direction	9
PART B — Lineage, and how datagraph builds its graph	12
Chapter 7 — What is data lineage?	12
Chapter 8 — Where the facts come from (the detective’s clues)	12
Chapter 9 — Reading Python code as a tree (AST)	13
Chapter 10 — Reading dbt’s manifest	14
Chapter 11 — Reading SQL with sqlglot: tables, then single columns	16
Chapter 12 — git diff: which lines did you change?	17
Chapter 13 — Gluing the fragments: name tags	18
Chapter 14 — Asking the question: impact, risk, tests, owners	18
Chapter 15 — The AI helper: explain, don’t build	19
Chapter 16 — Bugs we met, and what they taught	19
PART C — Tools, skills and your learning path	21
Chapter 17 — The tools and skills that were used	21
Chapter 18 — Your learning path (in order)	22
Glossary (all the words, in one place).....	22

(In Word: right-click the table and choose Update Field if page numbers are missing.)

How to read this book

This book has three parts. Part A teaches graphs from nothing, with drawings made of letters and arrows. Part B shows exactly how datagraph turns real project files into a graph and asks it “what breaks?”. Part C lists the tools and skills that were used and gives you a learning path.

Three kinds of boxes appear everywhere:

Big idea

A blue box is a key idea. If you remember only the blue boxes, you still understand the whole project.

Try it: a small exercise

A green box asks you to do something small on paper or in Python. Answers are at the back of the book.

Words to remember

An orange box collects the new words from a chapter, in plain language.

Code looks like this: `print("hello")`. You do not have to run the code to understand the book — but you can, every example is tiny.

PART A — Graphs from the very beginning

Chapter 1 — Dots and lines

Imagine you draw your four friends as dots on a page. Then you draw a line between two friends who sit next to each other in class. That drawing is a graph. That is really all a graph is: some things, and some connections between those things.

```

Asha  ----- Ben
|           |
Chen  ----- Dev
  
```

In graph language the things are called **nodes** (some books say vertices) and the connections are called **edges**. Asha, Ben, Chen and Dev are nodes. Each line is an edge. A graph with 4 nodes and 4 edges.

Graphs are everywhere once you start looking:

Where	Nodes (things)	Edges (connections)
A city map	places	roads between them
Your family tree	people	“is the parent of”
A recipe	ingredients and steps	“is needed for”
The internet	web pages	links
A data warehouse	tables and reports	“this report reads that table”
Computer code	files and functions	“this function calls that one”

Big idea 1

A graph = a list of things + a list of connections. Nothing more. The hard part is never the drawing; it is deciding what counts as a thing and what counts as a connection.

Try it: draw a graph

Draw five rooms in your home as dots. Draw a line between two rooms if there is a door between them. Count your nodes and your edges. Which room has the most lines touching it? That room is the ‘busiest’ node — remember this feeling, it comes back in Chapter 2.

Words to remember

node — a thing in the graph (a dot).

edge — a connection between two things (a line).

graph — the whole drawing: all nodes and all edges.

Chapter 2 — Arrows: when direction matters

Friends sitting next to each other is a two-way thing: if Asha sits next to Ben, Ben sits next to Asha. But many connections are one-way. “Asha gives her homework to Ben to check” is not the same as Ben giving his homework to Asha. For one-way connections we draw an arrow instead of a line.

```
Asha --> Ben --> Chen
              |
              v
              Dev
```

A graph with arrows is called a **directed graph**. Every edge now has a start (the tail) and an end (the arrow head). We write an edge as `src -> dst` — source to destination.

Counting arrows gives two useful numbers for each node. The **in-degree** is how many arrows point INTO it. The **out-degree** is how many arrows point OUT of it. In the drawing, Chen has in-degree 1 (from Ben) and out-degree 1 (to Dev). Dev has in-degree 1, out-degree 0.

Why care? A node with a big in-degree is one that many other nodes point to. In a data warehouse that is a table many reports read. If it changes, many things feel it. datagraph has a command called hotspots that finds exactly these nodes.

Big idea 2

One-way connections need arrows. Each node then has arrows coming in and arrows going out, and counting them already tells you which nodes are important.

Try it: arrows

Draw four people: You, Mum, Dad, Grandma. Draw an arrow from a person to someone they are the parent of. What is Grandma’s out-degree? What is your in-degree? (Answer at the back.)

Words to remember

directed graph — a graph whose edges are arrows (one-way).

src / dst — the start and the end of an arrow.

in-degree / out-degree — arrows coming in / going out of a node.

Chapter 3 — Paths, hops and “can I get there?”

Look at the arrow drawing again. From Asha you can go to Ben, then to Chen, then to Dev. That route — Asha → Ben → Chen → Dev — is called a **path**. Each arrow you follow is one **hop**. Dev is 3 hops away from Asha.

The most useful question in this whole book is: “Starting here, what can I reach by following arrows?” That is called **reachability**. From Asha you can reach Ben, Chen and Dev. From Dev you can reach nobody (no arrows go out). From Chen you can reach only Dev.

How far away something is — the number of hops on the shortest path — is called its **depth** or distance. Ben is at depth 1 from Asha, Chen at depth 2, Dev at depth 3. datagraph reports this number: “depth 1 = directly affected, depth 3 = affected through two other things”.

Loops (cycles) and no-loops (DAGs)

If you can start at a node, follow arrows and come back to where you started, the graph has a **cycle**. $A \rightarrow B \rightarrow A$ is a cycle. A graph with NO cycles is called a **DAG** — a Directed Acyclic Graph. Say it as “dag”. Data pipelines are DAGs: a report reads a table which reads raw data; the raw data never reads the report. Recipes are DAGs: you cannot need the cake to make the flour. But computer code can have cycles (a function calling itself, or two functions calling each other), so datagraph’s search must be careful not to go round and round forever. Chapter 5 shows the trick.

Big idea 3

“What breaks if I change X?” is the same question as “what can be reached from X by following arrows (in the right direction)?”. Reachability is the heart of impact analysis.

Try it: paths

In the Asha drawing, list every path that starts at Asha. Then tell me: is there a cycle? How many hops from Asha to Dev?

Words to remember

- path** — a route made of arrows.
- hop / depth** — one arrow / how many arrows away.
- reachable** — you can get there by following arrows.
- cycle** — a route that comes back to its start.
- DAG** — a directed graph with no cycles.

Chapter 4 — How a computer keeps a graph

A computer cannot see a drawing. It needs the graph written as a list. The simplest way is a notebook with one line per node, listing where its arrows go:

```
Asha -> [Ben]
Ben   -> [Chen]
Chen  -> [Dev]
Dev   -> []
```

This notebook is called an **adjacency list** (“adjacent” just means “next to”). In Python it is a dictionary — a box of labels, each label holding a list:

```
edges = {
    "Asha": ["Ben"],
    "Ben":  ["Chen"],
    "Chen": ["Dev"],
    "Dev":  [],
}
print(edges["Ben"])      # ['Chen']    – where can Ben go?
```

That is genuinely how graphs are stored. Libraries such as **networkx** (the one datagraph uses) wrap this notebook in a nice object with hundreds of ready-made helpers. Three lines do the same thing:

```
import networkx as nx
g = nx.DiGraph()           # Di = directed (arrows)
g.add_edge("Asha", "Ben")
g.add_edge("Ben", "Chen")
g.add_edge("Chen", "Dev")
print(list(g.successors("Ben"))) # ['Chen']
print(nx.descendants(g, "Asha"))  # {'Ben', 'Chen', 'Dev'} <- everything
                                # reachable!
```

datagraph uses **MultiDiGraph**. **Multi** means two nodes may have more than one arrow between them (for example a dbt model both “writes to” a table and “contains” a column — different arrows, different meanings). Each arrow carries a label saying what kind of connection it is. You will meet those labels in Chapter 6.

Big idea 4

A graph is stored as a notebook: for every node, the list of nodes its arrows reach. Python dictionaries and networkx are just tidy versions of that notebook.

Try it: the notebook

Write the adjacency list for your family-tree drawing from Chapter 2. Then, without drawing, answer: who can Grandma reach?

Words to remember

adjacency list — for each node, the list of its neighbours.

dictionary — Python’s “labelled boxes”.

networkx — a Python library full of graph helpers.

Chapter 5 — The ripple game: Breadth-First Search

Drop a stone in a pond. The first ring reaches the nearest leaves, the second ring the next ones, and so on. Breadth-First Search — BFS — is exactly that, on a graph. Start at one node; visit everything 1 hop away; then everything 2 hops away; then 3... and never visit the same node twice.

Let’s play it by hand on a slightly bigger graph (a data example now):

```
customer -> dim_customer -> fact_booking -> revenue_report
                        \
                        -> customer_dashboard
```

Ring	Who we look at	New nodes found (and their depth)
start	customer	customer is depth 0
ring 1	customer’s arrows	dim_customer (depth 1)
ring 2	dim_customer’s arrows	fact_booking (depth 2)
ring 3	fact_booking’s arrows	revenue_report, customer_dashboard (depth 3)
ring 4	nothing new to look at	done

Here is the same game as Python, with a comment on every line. Read it slowly — this little function is the engine of the whole project.

```
from collections import deque # a queue: first in, first out

def bfs(edges, start):
    depth = {start: 0}         # how far each node is; start = 0
    queue = deque([start])     # nodes whose arrows we still follow
    while queue:               # while there is work left...
        node = queue.popleft()  # take the oldest waiting node
        for nxt in edges.get(node, []): # each arrow out of it
            if nxt not in depth: # never seen before?
                depth[nxt] = depth[node] + 1 # one hop further
                queue.append(nxt)           # follow its arrows later
    return depth

edges = {"customer": ["dim_customer"],
        "dim_customer": ["fact_booking"],
        "fact_booking": ["revenue_report", "customer_dashboard"]}
```



```
print(bfs(edges, "customer"))
# {'customer': 0, 'dim_customer': 1, 'fact_booking': 2,
#  'revenue_report': 3, 'customer_dashboard': 3}
```

Look at the output. It is a list of every node that can be reached, with its depth. That is **exactly** what datagraph calls the blast radius. The real function is `ImpactGraph.impact()` in `graph/graph.py`; it is this function plus three extras: it asks a rule which direction to follow for each kind of arrow (Chapter 6), it has special handling for columns (Chapter 11), and it can stop at a maximum depth.

The line `if nxt not in depth` is the trick that makes cycles harmless: a node is visited once, so even $A \rightarrow B \rightarrow A$ cannot spin forever.

DFS, the cousin

Depth-First Search goes as deep as it can down one path before trying another — like exploring a maze by always taking the first turn. It is perfect for the question “show me every route from A to Z”. datagraph uses networkx’s `all_simple_paths` (a DFS) for the `paths` command.

Big idea 5

BFS = ripples. Visit neighbours ring by ring, never twice. The result — every reachable node with its distance — IS the blast radius of a change.

Try it: run the ripples

Copy the `bfs` function into Python. Add an arrow from `customer_dashboard` back to `customer` (a cycle!). Run it again. Does it still finish? Why?

Words to remember

BFS — breadth-first search; ring by ring.

queue — a waiting line: first in, first out.

visited — the set of nodes already seen (never visit twice).

DFS — depth-first search; one path to the end, then the next.

Chapter 6 — The domino secret: arrow direction vs impact direction

Here is the single most important idea in datagraph, and it trips up even experienced engineers. Stand three dominoes in a line and lean each one on the next:

A leans on B, B leans on C (drawn as arrows: $A \rightarrow B \rightarrow C$)

Now pull C out. What falls? B falls (it was leaning on C), and then A falls. Notice: the arrows point $A \rightarrow B \rightarrow C$, but the trouble travelled $C \rightarrow B \rightarrow A$ — the OPPOSITE way of the arrows!

Data and code are exactly like this. We write facts the natural way people say them: “dim_customer depends on customer” is the arrow $\text{dim_customer} \rightarrow \text{customer}$. But if customer changes, it is dim_customer that breaks. The change travels against the arrow.

Other facts travel with the arrow. “The file api.py contains the function handler” is $\text{api.py} \rightarrow \text{handler}$. Change the file and the function inside it is affected — same direction as the arrow.

So datagraph keeps a tiny rule table: for each kind of arrow, does impact go forward (with the arrow) or in reverse (against it)?

Kind of arrow (edge type)	Example	Impact travels
contains	file $\rightarrow$ function, model $\rightarrow$ column	forward — change the file, its functions are hit
calls	caller $\rightarrow$ callee	reverse — change the callee, callers are hit
imports	importer $\rightarrow$ imported	reverse
depends_on	dim_customer $\rightarrow$ customer	reverse — change upstream, downstream is hit
writes_to	model $\rightarrow$ table	forward
exposes	model $\rightarrow$ dashboard	forward

In the code this table is called `IMPACT_DIRECTION` in `graph/model.py`, and the BFS asks it for every arrow it meets (the helper is `_affected_neighbors`). Once you see it, you realise: the facts in the graph never lie about direction, and the rule table is the only place that knows how trouble spreads.

Big idea 6

Record facts the way people say them (arrows). Keep ONE rule per kind of arrow saying which way a change travels. Then one BFS gives the right answer for code and data at the same time.

Try it: dominoes

Draw: report $\rightarrow$ fact_table (the report depends on the table) and raw_file $\rightarrow$ loader_function (the file contains the function). Mark with a different colour the direction the trouble travels for each arrow.

Words to remember

forward / reverse — impact travels with / against the arrow.

upstream / downstream — where data comes from / where it goes to, like a river.

IMPACT_DIRECTION — datagraph's rule table.

PART B — Lineage, and how datagraph builds its graph

Chapter 7 — What is data lineage?

Lineage is a fancy word for “family tree”. Your lineage is your parents, their parents, and so on. Data lineage is the family tree of a number: where did this number on the dashboard come from?

```
raw.customers (a file from the shop system)
  | cleaned by the dbt model 'customer'
  v
analytics.customer ---> dim_customer ---> fact_booking ---> revenue_report
```

Reading upwards answers “where does this come from?” (useful when a number looks wrong). Reading downwards answers “who uses this?” — and THAT is what you need before you change something. datagraph is mostly interested in the downwards direction: if I change raw.customers, every table, model, function and dashboard below it is in danger.

Lineage comes in two sizes:

- **Table-level:** table A feeds table B. Good, but coarse: if you change one column of A, table-level lineage says ‘everything in B is in danger’.
- **Column-level:** column A.customer_id feeds column B.customer_key. Much sharper: changing A.email does not frighten B.customer_key. datagraph v0.2 does both, and Chapter 11 shows how.

Big idea 7

Lineage = the family tree of data. ‘What breaks?’ = reading the tree downwards from the thing you changed.

Words to remember

lineage — where data comes from and where it goes.

table-level / column-level — lineage between whole tables, or between single columns.

Chapter 8 — Where the facts come from (the detective’s clues)

A good detective does not guess; she collects clues that are already lying around. datagraph does the same: it never asks an AI “what do you think depends on what?”. It reads files that the project ALREADY has, and every one of them is a reliable witness:

Clue (artifact)	Who writes it	What it tells us
Python source files	the programmers	which functions exist, who calls whom, which files import which

Clue (artifact)	Who writes it	What it tells us
dbt manifest.json	the dbt tool, every time you run it	every model, source, dashboard (exposure), and exactly what each depends on
SQL files / compiled SQL	the data engineers	which tables a query reads, and which source column each output column comes from
git diff	git	which files — and which lines — you changed right now
OpenLineage events, DataHub lineage files	pipeline tools / the data catalog	lineage other systems already observed while running jobs
information_schema in the warehouse	the database itself	the real tables, columns and views that exist right now

Each clue is read by a small piece of code called an **extractor**. Every extractor does one thing: `extract()` -> `ImpactGraph` — read one kind of clue, return a small graph fragment. Then all fragments are merged into one big graph. Because every extractor is simple and separate, you can test it alone, and adding a new clue later (for example Airflow) is just one more extractor.

Big idea 8

Build the graph only from clues that already exist. One small extractor per clue. Merge the fragments. No guessing, no AI in this step — that is why the result can be trusted.

Words to remember

artifact — a file a tool or a person already produced.

extractor — a small reader that turns one artifact into graph nodes and edges.

deterministic — same input, same output, every time; no guessing.

Chapter 9 — Reading Python code as a tree (AST)

When you read “The cat sat on the mat”, your brain quietly splits it into pieces: a subject (the cat), a verb (sat), a place (on the mat). Computers do the same to code. Python has a built-in tool called `ast` (Abstract Syntax Tree) that turns code into a tree of pieces:

```
import db

def customers_endpoint():
    return db.load_customers()

# becomes a tree, roughly:
Module
```

```
|-- Import(db)
|-- FunctionDef(name='customers_endpoint', lineno=3, end_lineno=4)
    |-- Return
        |-- Call( func = db.load_customers )
```

The Python extractor walks this tree and writes down three kinds of facts:

1. **contains:** every FunctionDef or ClassDef becomes a node, with an arrow from the file: file:api.py → func:api.py::customers_endpoint. It also remembers the line numbers (3 to 4) — Chapter 12 needs them.
2. **imports:** every 'import db' becomes file:api.py → file:db.py (only if db.py is inside the project).
3. **calls:** every Call whose name matches a function defined in the project becomes an arrow caller → callee. This one is a guess by NAME (two different files could both define a load_customers), so the arrow is labelled inferred. You can tell datagraph to ignore inferred arrows with --no-inferred.

A bit of the real code, simplified:

```
tree = ast.parse(path.read_text(encoding="utf-8-sig")) # -sig: ignore a
Windows BOM mark
for node in ast.walk(tree):
    if isinstance(node, ast.FunctionDef):
        graph.add_node(Node(id=f"func:{rel}::{node.name}",
type=NodeType.FUNCTION, ...))
        graph.add_edge(Edge(src=f"file:{rel}",
dst=f"func:{rel}::{node.name}", type=EdgeType.CONTAINS))
```

Big idea 9

Code is data. The ast module turns a Python file into a tree you can walk, and every function, import and call you find becomes a node or an arrow.

Try it: see the tree

In Python: `import ast; print(ast.dump(ast.parse("def hi(): return 1"), indent=2))`. Find the FunctionDef and its name in the printout.

Words to remember

AST — the tree a computer builds from code.

FunctionDef / Import / Call — tree pieces for a function, an import, a call.

inferred — an arrow that is a smart guess, not a hard fact.

Chapter 10 — Reading dbt's manifest

dbt is a tool data teams use to build tables with SQL. Every time it runs, it writes a big file called `manifest.json` — a table of contents of the whole project. We do not need to parse any SQL to get the basics, because dbt already did the hard work. A tiny slice looks like this:

```
{
  "nodes": {
    "model.shop.dim_customer": {
      "resource_type": "model",
      "name": "dim_customer",
      "original_file_path": "models/dim_customer.sql",
      "database": "prod", "schema": "analytics",
      "columns": {"customer_key": {}, "email": {}},
      "meta": {"owner": "analytics-eng"},
      "depends_on": {"nodes": ["model.shop.customer"]},
      "compiled_code": "select customer_id as customer_key, email from
prod.analytics.customer"
    }
  },
  "sources": {...}, "exposures": {...}
}
```

From this one block the dbt extractor writes many facts:

Manifest field	Graph fact
name + resource_type	a node <code>dbt:dim_customer</code> of type <code>dbt_model</code>
depends_on.nodes	<code>dbt:dim_customer</code> → <code>dbt:customer</code> (<code>depends_on</code>) — the dbt DAG
original_file_path	<code>file:models/dim_customer.sql</code> → <code>dbt:dim_customer</code> (contains) — so a git change to the file maps to the model
database + schema + name	<code>dbt:dim_customer</code> → <code>table:prod.analytics.dim_customer</code> (<code>writes_to</code>)
columns	column: <code>dim_customer.customer_key</code> nodes, contained by the model
meta.owner / exposure owner	the owner stored on the node — so the report can say who to notify
exposures	<code>exposure:revenue_report</code> nodes with model → exposure (exposes)
compiled_code	real SQL → column-level lineage (Chapter 11)

Big idea 10

When a tool already wrote down the relationships (dbt's manifest), read them — don't rediscover them. It is faster and it is never wrong.

Words to remember

dbt — a tool that builds warehouse tables from SQL ‘models’.

manifest.json — dbt’s table of contents.

JSON — text written as nested boxes of labels and values.

exposure — dbt’s name for a dashboard or report that uses the data.

Chapter 11 — Reading SQL with sqlglot: tables, then single columns

SQL is the language used to ask databases for data. sqlglot is a Python library that reads SQL the way ast reads Python: it builds a tree. Take this statement:

```
CREATE TABLE analytics.dim_customer AS
SELECT c.customer_id AS customer_key, lower(c.email) AS email
FROM raw.customers c
```

Table-level lineage (easy)

sqlglot tells us the statement `CREATEs analytics.dim_customer` and the `SELECT` reads from `raw.customers`. So: `table:analytics.dim_customer → table:raw.customers (depends_on)`. The extractor simply collects every table name in the `SELECT`, skipping temporary names (CTEs).

Column-level lineage (the clever part)

Now the sharp question: which source column feeds `customer_key`? A human reads the `SELECT` and says “`c.customer_id`, and `c` is `raw.customers`”. sqlglot has a helper that does exactly this reading for us: `sqlglot.lineage.lineage("customer_key", select)`. It returns a tiny tree whose leaves are the source tables and columns, with every alias (`c`) and every CTE already untangled. The extractor walks to the leaves and writes:

```
column:analytics.dim_customer.customer_key ->
column:raw.customers.customer_id (depends_on)
column:analytics.dim_customer.email -> column:raw.customers.email
(depends_on)
```

Why is this a big deal? Because the column was `RENAMED`: `customer_id` became `customer_key`. A tool that matched columns by name would never connect them. Lineage by derivation (what was computed from what) connects them correctly. Now if someone changes `raw.customers.customer_id`, BFS walks (in reverse, Chapter 6) to `dim_customer.customer_key`, and from there to `fact_booking.customer_key`, and so on — and it does NOT frighten `dim_customer.email`, which is unrelated.

For dbt models the same helper runs on the `compiled_code` from the manifest, and the extractor maps table names back to model names, so column arrows run between dbt models (`column:dim_customer.customer_key → column:customer.customer_id`).

Two small safety rules for columns

- If a column has no lineage arrows (older manifest, no SQL), datagraph still propagates the change through the column's owning model to everything downstream, and flags same-named columns downstream. That is a heuristic, so those tree branches are marked inferred.
- A model change still lights up its own columns (model → column is contains, forward).

Big idea 11

Table lineage = which tables a query reads. Column lineage = which source column each output column is computed from. sqlglot works both out from the SQL itself — even through renames and aliases.

Try it: lineage by hand

SELECT a.id AS order_key, b.total * 2 AS double_total FROM orders a JOIN bills b ON ... — write the column lineage for order_key and double_total.

Words to remember

SQL — the language of databases.

sqlglot — a Python library that reads SQL into a tree and can trace columns.

alias — a nickname in SQL (c for raw.customers).

CTE — a temporary named query inside a bigger query (WITH c AS (...)).

Chapter 12 — git diff: which lines did you change?

git remembers every version of your files. 'git diff' shows what changed since the last saved version. datagraph runs two commands:

```
git diff --name-only      # which files changed? -> db.py
git diff --unified=0      # which lines? -> @@ -3,2 +3,3 @@ (new
lines 3 to 5 changed)
```

The second command prints 'hunk headers' like @@ -3,2 +3,3 @@. The part after the plus sign says: in the new file, starting at line 3, 3 lines are involved. The extractor turns that into a range, 3–5.

Remember Chapter 9 stored each function's line numbers (lineno 3 to end_lineno 4)? Now we just check overlap: does the changed range 3–5 touch the function's range 3–4? Yes → the function load_customers changed. Does it touch load_orders at lines 6–7? No → not changed. That is how datagraph says "you changed this exact function", not just "this file".

```
changed range:  3 ---- 5
load_customers: 3 -- 4      overlaps -> CHANGED
load_orders:    6 -- 7     no overlap -> untouched
```

Big idea 12

git tells us which lines changed; the AST told us which lines each function occupies; overlapping the two ranges finds the exact changed functions. That is the entry point of a pre-merge check.

Words to remember

git diff — show what changed.

hunk header — the @@ line that gives the changed line numbers.

overlap — two ranges share at least one line.

Chapter 13 — Gluing the fragments: name tags

Each extractor returns its own little graph. How do they join? By agreeing on names. Every node has an id made of a type word, a colon, and a path-like name:

file:models/customer.sql	func:src/api.py::customers_endpoint
dbt:dim_customer	source:raw.customers
table:prod.analytics.customer	column:dim_customer.customer_key
exposure:revenue_report	job:airflow/load_dim_customer

The dbt extractor says ‘file:models/customer.sql contains dbt:customer’. The git extractor, reading the diff, produces the id ‘file:models/customer.sql’. Same string → same node → the change flows into the model. Nobody had to teach the two extractors about each other; they only agreed on the name-tag format. That is why the id conventions are written down in the README as a rule.

The method `graph.merge(fragment)` adds every node and edge of a fragment to the big graph; if a node already exists, it just adds the new details to it. If an edge points at a node nobody has defined yet (dbt can depend on a source a later extractor will describe), a placeholder node is created so no arrow is ever lost.

Big idea 13

Fragments join because ids are built the same way everywhere. Agree on the name-tag rule once, and independent extractors automatically stitch into one graph.

Chapter 14 — Asking the question: impact, risk, tests, owners

Now the graph is built. Here is what happens when you type `datagraph impact dbt:customer`:

1. **Resolve** — turn your words into a node id. ‘customer’, ‘dbt:customer’ or ‘models/customer.sql’ all work; if two things match, the CLI lists the candidates.

2. **BFS** — the ripple game from Chapter 5, asking the direction table (Chapter 6) at every arrow. Result: every affected node with its depth.
3. **Risk score** — a points game. Each affected node is worth points by type (a dashboard 8, an API 6, a table or dbt model 3, a function or column 1). Nearer things count fully, far-away things at least half. Add a small bonus if many things are hit directly. Then: under 6 points LOW, under 18 MEDIUM, under 40 HIGH, otherwise CRITICAL. No AI, just arithmetic — so two people always get the same answer.
4. **Test plan** — rules per type: models → ‘dbt build --select model+’, columns → relationship/not-null tests, tables → schema checks, functions → unit tests, dashboards → look at them after deploy.
5. **Owners** — every affected node that knows its owner (from dbt meta, exposure owner, OpenLineage ownership facet) is collected into a ‘notify’ list.
6. **Report** — printed as a tree in the terminal (with ASCII fallback for old consoles), as JSON for machines, or as an interactive HTML page (datagraph.html).

Big idea 14

Impact = BFS. Risk = a transparent points game. Tests and owners = simple rules over the affected nodes. Everything is explainable because nothing is guessed.

Chapter 15 — The AI helper: explain, don’t build

Only after all of the above does AI appear. The finished analysis (the JSON with affected nodes, risk, owners, tests, trees) is sent to Claude with strict instructions: explain this to the engineer, walk the paths, say whether the risk feels right, list what to check before and after deploy — and never invent a node that is not in the JSON.

Think of it as a teacher reading a report card out loud and adding advice. The grades were computed elsewhere; the teacher does not change them. That is why datagraph can use AI for readability while the graph stays trustworthy. In the code this is `explain_impact()` in `ai/explain.py`; the MCP server and the `/datagraph` skill hand the same JSON to coding assistants such as Claude Code or Cursor.

Big idea 15

AI narrates a result it cannot alter. Trust comes from the deterministic graph; readability comes from the model.

Chapter 16 — Bugs we met, and what they taught

What went wrong	Why	The lesson
'models/customer.sql' was 'ambiguous'	Both the file node and the dbt model shared that path	Decide a priority order when resolving names (id, then name, then path, then the file node).
build found 0 Python files on Windows	Files saved by PowerShell start with an invisible BOM mark that ast.parse rejects; the error was swallowed	Read with utf-8-sig, and never let a parser fail silently. Test with files created the way real users create them.
The report crashed in Git Bash	That console cannot print  or  (cp1252 encoding)	Detect the console's abilities and fall back to plain ASCII. A CLI must never crash on output.
Changing a column reported NOTHING	Column nodes only had an arrow coming in; nothing flowed out	Every node type needs an answer to 'what happens if THIS changes?'. We added column propagation, then real column lineage.
rich ate the text [dbt]	Square brackets look like formatting codes to the rich library	Escape anything you print that you did not write yourself.

Big idea 16

Every bug above was found by USING the tool on an input the tests had not seen. Curiosity ("what if I change a different column?") is a testing tool.

PART C — Tools, skills and your learning path

Chapter 17 — The tools and skills that were used

Claude Code tools (how the assistant worked)

Tool / skill	What it did here
Write / Edit	Created the ~45 source, test and doc files, and made surgical fixes.
Bash / PowerShell	pip install, pytest, python -m build, creating clean virtual environments, git init/commit, running the CLI end-to-end, driving Microsoft Word to make PDFs.
WebFetch	Checked PyPI that the name 'datagraph' was free; read Graphify's README to compare features honestly.
WebSearch	Found out how and why Graphify became popular.
claude-api skill	Correct, current way to call the Claude API (model id, streaming, refusal handling) for the explain layer.
docx skill	Generated the project documentation and this learning book (docx-js → Word → PDF).
Memory	Saved a note about the project so future sessions start with context.

Programming skills inside the library

Skill	Where you see it	Why it matters
Python dataclasses & enums	graph/model.py	A clean, typed shape for nodes and edges.
Parsing with ast / json / sqlglot	extractors/	Turning files into facts.
Graph algorithms (BFS, paths, degrees) with networkx	graph/graph.py	The engine.
Testing with pytest (fixtures, tmp_path, monkeypatch, stubs)	tests/	Proof it works — offline, in 3 seconds, including the AI layer without an API key and a warehouse without a database.
CLI design with argparse	cli.py	Sub-commands, exit codes, JSON vs human output, graceful failure.
Terminal UI with rich	report.py	Trees and panels, with an ASCII fallback.
Packaging (pyproject, src layout, extras, entry point)	pyproject.toml	So 'pip install datagraph' just works.

Skill	Where you see it	Why it matters
git + GitHub Actions	.github/workflows, action.yml	Tests on every push; impact comments on every PR.
MCP server + skill file	mcp_server.py, skills/datagraph/SKILL.md	Lets AI coding assistants use the tool.
HTML/SVG/JS (no libraries)	html_report.py	A self-contained interactive picture of the blast radius.

Chapter 18 — Your learning path (in order)

1. **Week 1 — Python basics:** lists, dictionaries, functions, a little about classes. Goal: write the bfs function from Chapter 5 yourself from memory.
2. **Week 2 — Graphs:** redo every exercise in Part A; install networkx and reproduce the outputs. Goal: explain forward vs reverse impact to a friend with dominoes.
3. **Week 3 — Parsing:** print the AST of a small Python file; open a dbt manifest.json (or the one in examples/demo) and find depends_on by hand; run sqlglot.lineage on one SELECT.
4. **Week 4 — Read the real code:** graph/model.py (30 lines of ideas), graph/graph.py (impact, impact_tree), extractors/dbt_extractor.py. Use this book's chapters as the map.
5. **Week 5 — Testing and packaging:** run pytest, break a test on purpose and fix it, then build the wheel and install it in a clean virtual environment.
6. **Week 6 — Build something:** write a tiny new extractor (for example, read a CSV of 'report,table' pairs) and watch it merge into the graph. You now understand the whole project.

Answers to the exercises

- **Ch 2 family arrows:** Grandma → Mum (or Dad); Mum → You; Dad → You. Grandma's out-degree is 1 (or 2 if she is both grandparents' parent — fine either way); your in-degree is 2.
- **Ch 3 paths from Asha:** Asha→Ben, Asha→Ben→Chen, Asha→Ben→Chen→Dev. No cycle. Dev is 3 hops away.
- **Ch 4 who can Grandma reach:** everyone below her in the tree — Mum/Dad and You.
- **Ch 5 cycle:** it still finishes, because 'if nxt not in depth' refuses to visit customer a second time.
- **Ch 6 dominoes:** report → fact_table is depends_on, trouble travels fact_table ⇒ report (reverse). raw_file → loader_function is contains, trouble travels raw_file ⇒ loader_function (forward).
- **Ch 11 lineage:** order_key ← orders.id; double_total ← bills.total (the ×2 does not change where it comes from).

Glossary (all the words, in one place)

Word	Plain meaning
node	a thing in the graph
edge	a connection between two things; an arrow when directed
directed graph	a graph whose edges are one-way arrows
in-degree / out-degree	arrows coming into / going out of a node
path / hop / depth	a route of arrows / one arrow / how many arrows away
reachable	can be reached by following arrows
cycle / DAG	a route back to its start / a directed graph with no such routes
adjacency list	for each node, the list of its neighbours
BFS / DFS	ring-by-ring search / one-path-to-the-end search
blast radius	everything a change can affect (BFS result)
forward / reverse impact	a change travels with / against an arrow
upstream / downstream	where data comes from / goes to
lineage	the family tree of data, at table or column level
artifact / extractor	an existing file / the small reader that turns it into graph facts
deterministic	same input, same output, no guessing
AST	the tree a computer makes from source code
manifest.json	dbt's table of contents
sqlglot	a library that reads SQL and traces columns
git diff / hunk	what changed / a block of changed lines
inferred / extracted	a guessed arrow / an arrow read straight from an artifact
risk score	points per affected node, turned into LOW/MEDIUM/HIGH/CRITICAL
owner	the team or person responsible for an artifact
MCP / skill	ways to let AI assistants call a tool